

DESARROLLO DE APLICACIONES WEB ENTORNO SERVIDOR:

DIARIO DE PRÁCTICAS

Alumno: Fara Santeyana, María Guillermina.

NIA: 13298631. **Curso:** 2do DAW.

Instituto: I.E.S La Sènia.

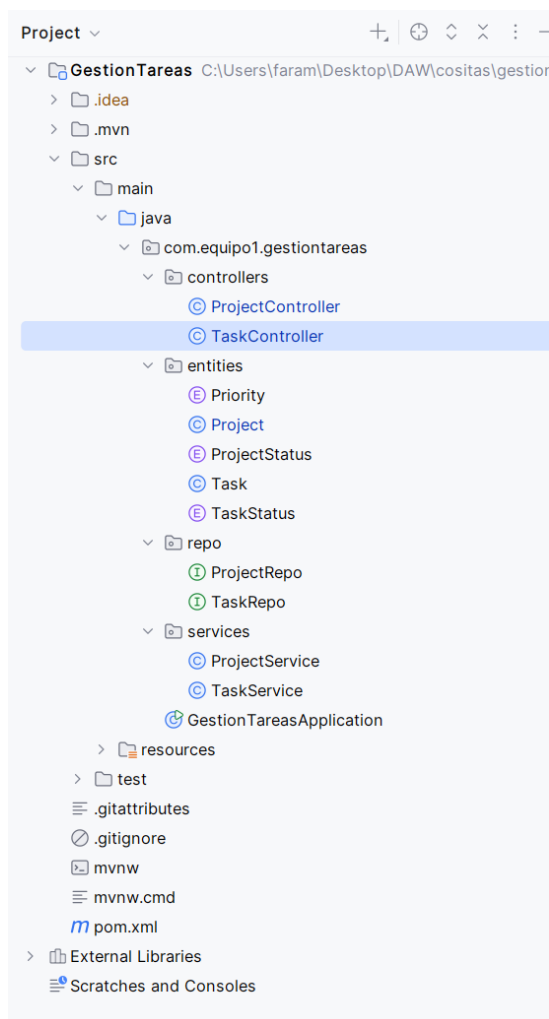
Año: 2025-2026.

	Descripción actividad	Orientaciones	Observaciones
Lunes 09/03	Presentación firma de contratos, conocer la empresa y equipos. Entrega de portátiles instalación y configuración de herramientas de trabajo. Dinámica kanban con planteamiento de tareas	Organización y presentación	
Martes 10/03	Repaso de comandos de Linux y bash Repaso intensivo de git, merge request y creación de contenedores Docker.	Git, docker, bash.	Repaso y nivelación de conocimientos
Miércoles 11/03	Trabajo en equipo para conocer el nivel de los integrantes. Creación de proyecto springboot, h2, Jpa. Modelo-Vista-Controlador. Creación de API. Endpoints. Vinculación con frontend JS. Control de versiones gitlab, resolución de conflictos.	Springboot, java, javascript, h2, Jpa, git. MVC. API. Endpoints.	Pequeño proyecto grupal de 3 integrantes para conocer el nivel y observar trabajo en grupo. 2 para el back y 1 para el front.
Jueves 12/03	Explicación conceptos de principios SOLID, Clean Code, Diseño simple. Ejercicios usando estas modalidades. Explicación de metodologías AGILE, YouTrack.	pilares para mantener código legible, mantenible, flexible y escalable.	Ejercicios prácticos para aplicar.
Viernes 13/03	Explicación y prácticas de metodologías de eXtreme Programming y calidad. Prácticas aplicando TDD Pair Programming.	Metodología ágil de desarrollo de software centrada en la calidad, retroalimentación continua, para mejorar la eficiencia	Trabajos en equipos para aplicar las metodologías aprendidas.

Código aplicado miércoles 11/03

Se nos planteó la siguiente actividad para conocer el nivel con el que contamos (no podíamos usar IA's, claro) y para conocer como trabajamos en equipo. Los puntos a trabajar eran: arquitectura, backend y frontend, se nos dejó a elección las tecnologías a usar. Nosotros escogimos, arquitectura MVC; java con springboot, endpoints, sql bases de datos; para el frontend optamos por JavaScript Vanilla (Código del frontend no lo tengo porque olvidé clonar el repositorio).

Estructura del proyecto:

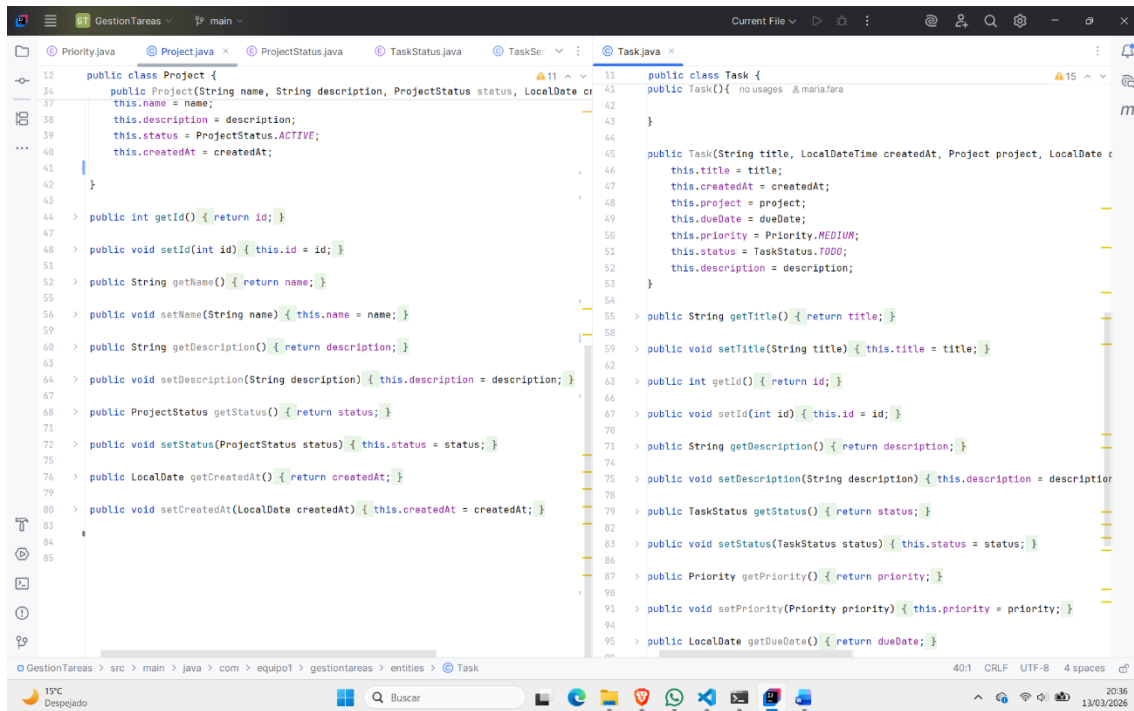


Primeras Inserciones

```
1 INSERT INTO project (name, description, status, created_at)
2 VALUES ('Configuración red local', 'Configuración de red de internet local', 0, CURRENT_DATE);
3
4 INSERT INTO project (name, description, status, created_at)
5 VALUES ('Instalar y configurar ordenadores', 'Instalar todas las herramientas', 0, CURRENT_DATE);
6
7 INSERT INTO task (title, description, status, priority, due_date, project_id, created_at)
8 VALUES ('Reiniciar router', 'Desconectar el router del toma corrientes', 0, 1, DATE '2026-10-10', 1, CURRENT_TIMESTAMP);
9
10 INSERT INTO task (title, description, status, priority, due_date, project_id, created_at)
11 VALUES ('Cambiar la contraseña', 'Configurar una contraseña segura', 0, 1, DATE '2026-10-10', 1, CURRENT_TIMESTAMP);
12
13 INSERT INTO task (title, description, status, priority, due_date, project_id, created_at)
14 VALUES ('Poner en modo bridge', 'Configurar modo puente el router', 0, 1, DATE '2026-10-11', 1, CURRENT_TIMESTAMP);
15
16 INSERT INTO task (title, description, status, priority, due_date, project_id, created_at)
17 VALUES ('Conectar Switch', 'Vincular switch y router', 0, 1, DATE '2026-10-12', 1, CURRENT_TIMESTAMP);
18
19 INSERT INTO task (title, description, status, priority, due_date, project_id, created_at)
20 VALUES ('Descargar aplicaciones', 'Descargar aplicaciones necesarias con make me admin', 0, 1, DATE '2026-03-11', 2, CURRENT_TIMESTAMP);
21
22 INSERT INTO task (title, description, status, priority, due_date, project_id, created_at)
23 VALUES ('Configurar aplicaciones', 'Configurar aplicaciones, cuentas y versiones', 0, 1, DATE '2026-03-11', 2, CURRENT_TIMESTAMP);
```

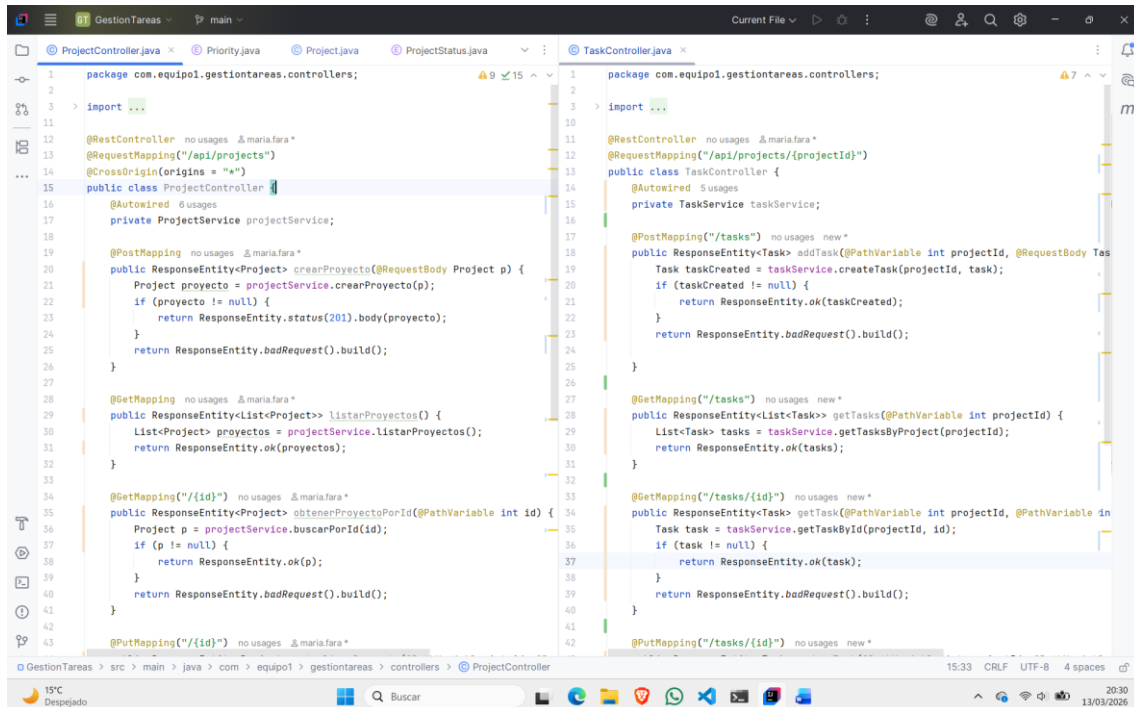
Modelo, entidades

```
4 import ...
9
10 @Entity 32 usages & maria.fara
11 @Table(name = "project")
12 public class Project {
13     @Id 2 usages
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     @Column(name = "projectId", nullable = false)
16     private int id;
17
18     @Column(name = "name", nullable = false) 3 usages
19     @Size(min = 3, max = 100, message = "the name must be between 3 and 100 characters")
20     private String name;
21
22     @Column(name = "description") 3 usages
23     private String description;
24
25     @Column(name = "status", nullable = false) 3 usages
26     private ProjectStatus status;
27
28     @Column(name = "createdAt", nullable = false) 3 usages
29     private LocalDate createdAt;
30
31     public Project() { no usages & maria.fara
32     }
33
34     public Project(String name, String description, ProjectStatus status, LocalDate createdAt) {
35     }
36
37     {
38         this.name = name;
39         this.description = description;
40         this.status = ProjectStatus.ACTIVE;
41         this.createdAt = createdAt;
42     }
43
44 package com.equipo1.gestionareas.entities;
45
46 import ...
47
48 @Entity 25 usages & maria.fara
49 @Table(name = "task")
50 public class Task {
51     @Id 2 usages
52     @GeneratedValue(strategy = GenerationType.IDENTITY)
53     @Column(name = "taskId", nullable = false)
54     private int id;
55
56     @Size(min = 3, max = 200, message = "the name must be between 3 and 300")
57     @Column(name = "title", nullable = false)
58     private String title;
59
60     @Column(name = "description", nullable = false) 3 usages
61     private String description;
62
63     @Column(name = "status", nullable = true) 3 usages
64     private TaskStatus status;
65
66     @Column(name = "priority", nullable = false) 3 usages
67     private Priority priority;
68
69     @Column(name = "dueDate", nullable = true) 3 usages
70     private LocalDate dueDate;
71
72     @JoinColumn(name = "projectId", nullable = false) 3 usages
73     @ManyToOne
74     private Project project;
75
76     @Column(name = "createdAt", nullable = true) 3 usages
77     private LocalDateTime createdAt;
```



```
12 public class Project {
13     public Project(String name, String description, ProjectStatus status, LocalDate createdAt) {
14         this.name = name;
15         this.description = description;
16         this.status = ProjectStatus.ACTIVE;
17         this.createdAt = createdAt;
18     }
19
20     public int getId() { return id; }
21
22     public void setId(int id) { this.id = id; }
23
24     public String getName() { return name; }
25
26     public void setName(String name) { this.name = name; }
27
28     public String getDescription() { return description; }
29
30     public void setDescription(String description) { this.description = description; }
31
32     public ProjectStatus getStatus() { return status; }
33
34     public void setStatus(ProjectStatus status) { this.status = status; }
35
36     public LocalDate getCreatedAt() { return createdAt; }
37
38     public void setCreatedAt(LocalDate createdAt) { this.createdAt = createdAt; }
39 }
40
41 public class Task {
42     public Task() { no usages }
43
44     public Task(String title, LocalDateTime createdAt, Project project, LocalDate dueDate, Priority priority, TaskStatus status, String description) {
45         this.title = title;
46         this.createdAt = createdAt;
47         this.project = project;
48         this.dueDate = dueDate;
49         this.priority = Priority.MEDIUM;
50         this.status = TaskStatus.TODO;
51         this.description = description;
52     }
53
54     public String getTitle() { return title; }
55
56     public void setTitle(String title) { this.title = title; }
57
58     public int getId() { return id; }
59
60     public void setId(int id) { this.id = id; }
61
62     public String getDescription() { return description; }
63
64     public void setDescription(String description) { this.description = description; }
65
66     public TaskStatus getStatus() { return status; }
67
68     public void setStatus(TaskStatus status) { this.status = status; }
69
70     public Priority getPriority() { return priority; }
71
72     public void setPriority(Priority priority) { this.priority = priority; }
73
74     public LocalDate getDueDate() { return dueDate; }
75 }
```

Controlador



```
1 package com.equipo1.gestionareas.controllers;
2
3 import ...
4
5 @RestController no usages & maria.fara *
6 @RequestMapping("/api/projects")
7 @CrossOrigin(origins = "*")
8 public class ProjectController {
9     @Autowired 0 usages
10     private ProjectService projectService;
11
12     @PostMapping no usages & maria.fara *
13     public ResponseEntity<Project> crearProyecto(@RequestBody Project p) {
14         Project proyecto = projectService.crearProyecto(p);
15         if (proyecto != null) {
16             return ResponseEntity.status(201).body(proyecto);
17         }
18         return ResponseEntity.badRequest().build();
19     }
20
21     @GetMapping no usages & maria.fara *
22     public ResponseEntity<List<Project>> listarProyectos() {
23         List<Project> proyectos = projectService.listarProyectos();
24         return ResponseEntity.ok(proyectos);
25     }
26
27     @GetMapping("/{id}") no usages & maria.fara *
28     public ResponseEntity<Project> obtenerProyectoPorId(@PathVariable int id) {
29         Project p = projectService.buscarPorId(id);
30         if (p != null) {
31             return ResponseEntity.ok(p);
32         }
33         return ResponseEntity.badRequest().build();
34     }
35
36     @PutMapping("/{id}") no usages & maria.fara *
37 }
38
39 package com.equipo1.gestionareas.controllers;
40
41 import ...
42
43 @RestController no usages & maria.fara *
44 @RequestMapping("/api/projects/{projectId}")
45 public class TaskController {
46     @Autowired 5 usages
47     private TaskService taskService;
48
49     @PostMapping("/tasks") no usages new *
50     public ResponseEntity<Task> addTask(@PathVariable int projectId, @RequestBody Task task) {
51         Task taskCreated = taskService.createTask(projectId, task);
52         if (taskCreated != null) {
53             return ResponseEntity.ok(taskCreated);
54         }
55         return ResponseEntity.badRequest().build();
56     }
57
58     @GetMapping("/tasks") no usages new *
59     public ResponseEntity<List<Task>> getTasks(@PathVariable int projectId) {
60         List<Task> tasks = taskService.getTasksByProject(projectId);
61         return ResponseEntity.ok(tasks);
62     }
63
64     @GetMapping("/tasks/{id}") no usages new *
65     public ResponseEntity<Task> getTask(@PathVariable int projectId, @PathVariable int id) {
66         Task task = taskService.getTaskById(projectId, id);
67         if (task != null) {
68             return ResponseEntity.ok(task);
69         }
70         return ResponseEntity.badRequest().build();
71     }
72
73     @PutMapping("/tasks/{id}") no usages new *
74 }
```

```
15 public class ProjectController {
31     return ResponseEntity.ok(projectos);
32 }
33
34 @GetMapping("/{id}") no usages & maria.fara *
35 public ResponseEntity<Project> obtenerProyectoPorId(@PathVariable int id) {
36     Project p = projectService.buscarPorId(id);
37     if (p != null) {
38         return ResponseEntity.ok(p);
39     }
40     return ResponseEntity.badRequest().build();
41 }
42
43 @PutMapping("/{id}") no usages & maria.fara *
44 public ResponseEntity<Project> actualizarProyecto(@PathVariable int id, @RequestBody Project proyecto) {
45     Project proyecto = projectService.actualizarProyecto(p, id);
46     return ResponseEntity.ok(proyecto);
47 }
48
49 @DeleteMapping("/{id}") no usages & maria.fara *
50 public ResponseEntity<Void> eliminarProyecto(@PathVariable int id) {
51     projectService.eliminarProyecto(id);
52     return ResponseEntity.noContent().build();
53 }
54
55 @PatchMapping("/{id}/archive") no usages & maria.fara *
56 public ResponseEntity<Project> archivar(@PathVariable int id) {
57     Project proyecto = projectService.archivar(id);
58     return ResponseEntity.ok(proyecto);
59 }
60
61
62
63
}

13 public class TaskController {
29     List<Task> tasks = taskService.getTasksByProject(projectId);
30     return ResponseEntity.ok(tasks);
31 }
32
33 @GetMapping("/tasks/{id}") no usages new *
34 public ResponseEntity<Task> getTask(@PathVariable int projectId, @PathVariable int taskId) {
35     Task task = taskService.getTaskById(projectId, id);
36     if (task != null) {
37         return ResponseEntity.ok(task);
38     }
39     return ResponseEntity.badRequest().build();
40 }
41
42 @PutMapping("/tasks/{id}") no usages new *
43 public ResponseEntity<Task> updateTask(@PathVariable int projectId, @PathVariable int taskId, @RequestBody Task taskUpdated) {
44     Task taskUpdated = taskService.updateTask(projectId, id, task);
45     if (taskUpdated != null) {
46         return ResponseEntity.ok(taskUpdated);
47     }
48     return ResponseEntity.badRequest().build();
49 }
50
51 @DeleteMapping("/tasks/{id}") no usages new *
52 public ResponseEntity<Void> deleteTask(@PathVariable int projectId, @PathVariable int taskId) {
53     taskService.deleteTask(projectId, id);
54     return ResponseEntity.noContent().build();
55 }
56
57
58
}
```

Implementaciones

```
1 package com.equipo1.gestionareas.services;
2
3 import ...
4
5 @Service 2 usages & maria.fara
6 public class ProjectService {
7     @Autowired 6 usages
8     private ProjectRepo projectRepo;
9     private TaskRepo taskRepo; no usages
10
11     public Project buscarPorId(int id) { return projectRepo.findById(id).orElse(null); }
12
13     public Project crearProyecto(Project p) { 1 usage & maria.fara
14         if (p != null) {
15             projectRepo.save(p);
16             return p;
17         }
18         return null;
19     }
20
21     public Project actualizarProyecto(Project p, int id) { 1 usage & maria.fara
22         Project pId = projectRepo.findById(id).orElse(null);
23         if (pId != null) {
24             pId.setId(p.getId());
25             return p;
26         }
27         return null;
28     }
29
30     public void eliminarProyecto(int id) { projectRepo.deleteById(id); }
31
32     public Project archivar(int id) { 1 usage & maria.fara
33         Project p = buscarPorId(id);
34         if (p != null) {
35             p.setStatus(ProjectStatus.ARCHIVED);
36             return projectRepo.save(p);
37         }
38     }
39 }

1 package com.equipo1.gestionareas.services;
2
3 import ...
4
5 @Service 2 usages & maria.fara
6 public class TaskService {
7     @Autowired 6 usages
8     private TaskRepo taskRepo;
9     @Autowired 1 usage
10     private ProjectRepo projectRepo;
11
12     public Task createTask(int projectId, Task task) { 1 usage & maria.fara
13         Project project = projectRepo.findById(projectId)
14             .orElseThrow(() -> new RuntimeException("Project not found"));
15         task.setProject(project);
16         return taskRepo.save(task);
17     }
18
19     public List<Task> getTasksByProject(int projectId) { return taskRepo.findByProjectId(projectId); }
20
21     public Task getTaskById(int projectId, int id) { 2 usages & maria.fara
22         Task task = taskRepo.findById(id)
23             .orElseThrow(() -> new RuntimeException("Task not found"));
24         if (task.getProject().getId() != projectId) {
25             throw new RuntimeException("Task does not belong to this project");
26         }
27         return task;
28     }
29
30     public Task updateTask(int projectId, int id, Task updatedTask) { 1 usage & maria.fara
31         Task task = taskRepo.findByIdAndProjectId(id, projectId)
32             .orElseThrow(() -> new RuntimeException("Task not found"));
33         task.setStatus(updatedTask.getStatus());
34         return taskRepo.save(task);
35     }
36
37     public void deleteTask(int projectId, int id) {
38         Task task = taskRepo.findByIdAndProjectId(id, projectId)
39             .orElseThrow(() -> new RuntimeException("Task not found"));
40         taskRepo.delete(task);
41     }
42 }


```

